PA1

SKK UNIVERSITY

PA1

- **Purpose**
  - Learn Unix File I/O and data structures
- **Goal**
  - Write a program that can perform basic search operations on a text file with an arbitrary size
  - The name of the file is passed as a command-line argument.
  - When the program starts running, it waits for the user to enter a query, and the program then performs a search operation based on the query.
- **Deadline: 2026-04-05 (Sun) 23:59**

Search operations

1 Find all occurrences of a single word

- **Query:** Contains only a single word.
- **Action:** Search for all occurrences of the word in the given file.
- **Format:** `"[line number]:[start index of the word]"`
- **Example:**
  - » **1 | The** quick brown fox jumps over **the** lazy dog.
  - » **2 | The** five boxing wizards jump quickly.

  - » **Query:** The
  - » **Expected Output:**
    - » 1:0 1:31 2:0

Search operations

2 Find lines that contain multiple words

- **Query:** Contains multiple words separated with a single space (no quotation marks, asterisks).
- **Action:** Search for lines containing the word in the given file, ignoring word order.
  - » Query "`[word1] [word2]`" accepts lines that contain "`[word1] [word2]`" or "`[word2] [word1]`".
- **Format:** **`"[line number]"`**
- **Example:**
  - » **1 |** I **think** therefore I **am**.
  - » **Query:** am think
  - » **Expected Output:**
    - » 1

Search operations

3 Find all occurrences of phrases

- **Query:** Contains multiple words wrapped in quotation marks.
- **Action:** Search for all occurrences of the phrase in the given file while considering word order.
  - » No line breaks or quotation marks in phrase, but may contain consecutive spaces or tabs (i.e., **`“[first] [last]”`** != **`“[first]  [last]”`**)
- **Format:** **`"[line number]:[start index of the word]“`**
- **Example:**
  - » **1 | `HA HA HA HA`**
  - » **Query:** HA HA
  - » **Expected Output:** “`1:0 1:3 1:6`”

Search operations

4 Find lines that match a basic pattern

- **Query:** A query where two words are separated by an asterisk (*****) and no spaces in between.
- **Action:** Search for lines that match the pattern in the given file
  - » (i.e., should contain the two words and there may be zero or more words in between)
- **Format:** `"[line number]"`
- **Example:**
  - » `1 | I am a student.`
  - » `2 | I really am a student.`
  - » **Query:** I*am
  - » **Expected Output:** “1 2”

Definitions

- **Word: strings separated by spaces; case-insensitive**
  - (e.g., **`tom, and, mckenzie, brother's, priests', wide-awake, sons', score:`**)
  - Words containing different characters, including non-alphanumeric characters, are considered different words. Words with different case match.
    - » Searching "**I**" should only check the occurrences of "I" and "i" and not those of "I'm", "I!" and "Ice".
- **Phrase: one or more words arranged in a specific order**
  - Whitespace counts as a single character. Phrases with varying number of spaces do not match with each other (i.e., **`"[first] [last]"`** and **`"[first]  [last]"`** are two distinct phrases.
- **Pattern: a sequence of multiple words containing an asterisk**
- **Line number: index of the line that contains the keyword.**
  - **Empty lines are counted.**
  - Starts from 1
- **Start index of the word: the index of the first character of the word on the line**
  - **Whitespace (except for \n) and non-alphanumeric characters are counted.**
  - Starts from 0
- **Query: the search term(s) passed by the user.**
  - Should have an associated output for each query even if the result/output is empty!

Note on input

- **For inputs with * and “**
  - There are no inputs with both an asterisk (*) and a quotation mark (“).
  - Inputs with * are handled with 4.
  - Inputs with “ are handled with 3.
  - Inputs with no * and “ but have multiple words are handled with 2.
  - Inputs with only one word and no * and “ are handled with 1.
- **The maximum length of the input is 4,096 characters.**
- **Punctuation is part of the word.**
  - i.e,. Input “`word`” does not match with “`word!`” and “`word?`”

    Input “`word!`” matches with “`word!`” but not “`word`”
- **The characters in the input are ASCII characters.**
- **Program waits for input indefinitely and exits when the input is “`PA1EXIT`”.**

Printing output

- **Always print to standard output.**
- **If the requested input occurs multiple times in the file,**
  - 1, 3: print **all locations** sequentially even if they are in the same line.
    - » (i.e., It should be 1:0 1:2 4:8 6:13 9:8 and not 9:8 1:2 4:8 1:0 6:13)
  - 2, 4: print the line number **only once** even if there are multiple instances.
  - Locations/Line numbers should be separated by a single space, and each search output should be separated by a new line.

```
$ "he is"

1955:33 2315:42 5885:22 ↵
```

  - Note that there is a space between the last occurrence and newline for simplicity.
  - The locations/line numbers should be printed sequentially.
- **An output is always associated with an input query, even when there are no occurrences found for the query.**
  - If there is no result, the output should be empty (i.e., "**1:0** \n\n**1:0** \n").
  - This means the first line of stdout should be the answer to the first query and so on.

Notes on the input text file

- **The filename of the input text file is passed as a command line argument (i.e. `argv[1]`).**
- **Characters in the text files are ASCII characters.**
- **The file has an arbitrary length that is much larger than the system memory.**
  - This means that the file cannot be loaded into memory.
- **However, the lines and the words in the lines have an arbitrary length that is smaller than system memory.**

Scoring Policy

- It is based on a 100-point scale and must follow the format described above, otherwise no points will be given.
- Ten points are deducted per day after the deadline (max 4 days).
- Score depends on whether the test case passed or not.
- Several input files are used for scoring, including the input file in the example. These input files may exceed 10 GB.
- Test cases should take around 10 seconds to complete. If the program takes more than 5 minutes to complete a test case, no points will be given for that test case.
- The memory of the scoring server is assumed to be 1 GB.

Restrictions

- **Restrictions**
  - Project must be compiled and executed in a Linux environment.
  - You may get a **0** if you use headers files not found here.
    - » At least one hand-implemented header file (required)
    - » **`unistd.h fcntl.h stdlib.h sys/types.h sys/stat.h errno.h stdint.h`**
  - This is a personal assignment.
    - » You can discuss the task together, but you must write the source code by yourself.
  - The Makefile should create a binary called **pa1**
    - » **If not, you may get a 0.**

Submission

- Project directory should look like this:

```
pa1 (directory)
- *.c (C code files)
- *.h (Header files)
- Makefile
```

- Compress the pa1 directory into a tarball with the format “STUDENTID_pa1.tar.zst” and submit the tarball to iCampus before the deadline.
- Deadline: 2026-04-05 (Sun) 23:59

Example

```
$ ./pa1 500-Days-of-Summer_s.txt
500 days
6 6419
he is
50 1039 1822 1955 2256 2315 3494 3503 4353 4360 4445 4831 5101 5885 6325
"he is"
1955:33 2315:42 5885:22
he*is
2315 4445 5101
she he
1370 1512 1513 3423 3473 3478 3550 4255 4510 4515 5413 5672 6154 6188
loved
106:30 1122:9 1150:24 3961:25 4739:17 5921:28

…
```

Example

```
$ ./pa1 500-Days-of-Summer_s.txt > result.out
he
hey
she
tom
summer is
_summer is"
landscape
together
we*her
no much
immediately
no*much
quarterback
we
PA1EXIT

$ diff -bsq result.out answer.out
Files result.out and answer.out are identical
$ diff result.out answer.out
$
```

PA1

SKK UNIVERSITY

PA1

- **주제**
  - Unix 파일의 입출력 및 데이터 구조의 이해
- **목표**
  - 임의 크기의 텍스트 파일에 대해 기본적인 검색 작업을 수행할 수 있는 프로그램 작성
  - 대상 파일의 이름은 명령행 인자로부터 전달받아야 함
  - 프로그램이 실행되면, 사용자로부터 검색어를 입력 받고 해당 검색어에 대한 검색 작업을 수행할 수 있어야 함

- **마감일: 2026년 4월 5일 (일) 23:59**

Search operations

1 단일 단어 입력 시, 등장하는 위치 전부 표시

- 사용자로부터 **하나의 단어로 된 검색어**를 입력 받는 경우, 해당 단어가 파일 내에 등장하는 모든 위치를 표시할 수 있어야 함
- 각 등장 위치에 대해 "**[줄 번호]:[해당 줄에서의 시작 위치]**" 형식으로 **표준 출력(STDOUT)**에 표시할 수 있어야 함
- 예: "1:0 1:5 1:10" 은 1번째 줄에서 단어가 0, 5, 10번째 위치에 나타난다는 의미

Search operations

2 다중 단어 입력 시, 등장하는 줄 번호 표시

- 사용자로부터 **공백(스페이스 바)으로 구분된 여러 단어(따옴표는 포함시키지 않음)**를 입력받는 경우, 프로그램은 입력된 단어들이 동시에 포함되어있는 줄을 출력할 수 있어야 함
- 단어의 순서는 신경쓰지 않음
  - » 예를 들어 사용자가 ‘단어1’, ‘단어2’를 입력하면, ‘단어1’, ‘단어2’ 또는 ‘단어2’, ‘단어1’이 포함된 줄을 모두 찾는다.
- 각 입력에 대한 결과는 해당 줄의 번호만 “**[줄 번호]**” 형식으로 **표준 출력(STDOUT)**에 표시할 수 있어야 함
- 예: 사용자가 “am think”를 입력했고, 파일의 첫 번째 줄에 등장하는 문장이 “I think therefore I am”이라면, 출력은 “1”이 됨

Search operations

3 구(문장 덩어리) 입력 시, 등장하는 위치 전부 표시

- 사용자로부터 **따옴표(”)로 묶인 여러 단어(구)**를 입력받는 경우, 프로그램은 해당 구가 파일 내에 등장하는 모든 위치를 검색해야 함
- 구 내부에는 줄바꿈(\n)이나 추가적인 따옴표(”)는 포함될 수 없지만, 여러 개의 연속된 공백(스페이스)이나 탭 문자는 포함될 수 있음
  - » 예: "first last" 와 "first last"는 일치하지 않는다.
- 시작 따옴표와 첫 단어 사이, 마지막 단어와 끝 따옴표 사이에는 공백이 없음
  - » 예: **“[first] . . . [last]”**
- 각 입력에 대한 결과는 **“[줄 번호]:[해당 줄에서의 시작 위치]”** 형식으로 **표준 출력(STDOUT)**에 표시할 수 있어야 함
- 같은 줄에 구가 여러 번 등장하는 경우, 모두 순서대로 출력
- 예: 파일의 첫 줄이 "HA HA HA HA"이고, 사용자가 "HA HA"를 입력했다면 출력은 "1:0 1:3 1:6"이 됨.

Search operations

4 기본 패턴에 해당하는 줄 찾기

- 사용자로부터 **별표(*)가 단어 두 개 사이에 위치한 문자열**을 입력받은 경우, 프로그램은 해당 패턴이 포함된 줄을 검색해야 함
- 입력 형식은 별표 앞뒤로 공백 없이 단어가 붙은 형태여야 하며 (예: hello*world), 공백이 포함된 입력은 허용되지 않음
- 패턴 내 단어의 순서는 중요하며, 예를 들어 [word1]*[word2]를 입력한 경우 [word2]*[word1]은 검색 대상에 포함되지 않음.
- 각 입력에 대한 결과는 해당 패턴이 포함된 줄 번호를 [줄 번호] 형식으로 **표준 출력(STDOUT)**에 표시할 수 있어야 함
- 같은 줄에 패턴이 여러 번 등장해도 줄 번호는 한 번만 출력해야 함
- 예: 파일의 첫 줄이 "HA HA HA HA"이고, 사용자가 HA*HA를 입력했다면 출력은 "1"이 됨

Definitions

- **단어 (Word): 공백으로 구분된 문자열로, 대소문자를 구분하지 않음**
  - 예: tom, and, mckenzie, brother's, priests', wide-awake, sons', score:
  - 알파벳 이외의 문자를 포함한 단어는 서로 다른 단어로 간주함
    - » 같은 단어라도 "I"를 검색하면 "I", "i"만 일치하며, "I'm", "I!", "Ice"는 일치하지 않음
- **구 (Phrase): 하나 이상의 단어가 특정 순서로 나열된 문자열**
  - 구 내 공백은 하나의 문자로 취급되며, 공백 개수가 다르면 서로 다른 구로 간주함
  - 예: "first last"와 "first last"는 서로 다른 구로 처리됨
- **패턴 (Pattern): 별표(*)가 포함된 두 단어 이상의 조합**
- **줄 번호 (Line number): 검색어가 포함된 줄의 인덱스**
  - **빈 줄도 줄 번호에 포함되며**
  - **번호는 1부터 시작함**
- **단어 시작 인덱스 (Start index of the word): 해당 줄에서 단어가 시작되는 첫 문자 위치**
  - **줄 내의 공백 문자(\n 제외) 및 특수문자도 인덱스에 포함되며**
  - **인덱스는 0부터 시작함**
- **검색어 (Query): 사용자가 입력한 검색어**
  - 검색 결과가 없더라도, 모든 검색어에 대해 출력 결과가 존재해야 함

Note on input

- *** 또는 "이 포함된 입력 처리 방식**
  - 별표(*)와 따옴표(")가 동시에 포함된 입력은 허용되지 않음
  - 별표(*)가 포함된 입력은 4 기본 패턴 처리 방식으로 처리함
  - 따옴표(")가 포함된 입력은 3 구(문장 덩어리) 처리 방식으로 처리함
  - 별표(*)와 따옴표(")가 없고, 단어가 여러 개인 경우는 2 다중 단어 처리 방식으로 처리함
  - 단어가 하나이고 별표(*)와 따옴표(")가 포함되지 않은 경우는 1 단일 단어 처리 방식으로 처리함
- **입력 가능한 최대 길이는 4,096자**
- **문장 부호도 단어의 일부로 간주됨**
  - 예: "word"는 "word!" 또는 "word?"와 일치하지 않음
    "word!"는 "word!"와 일치하지만 "word"와는 일치하지 않음
- **입력에 사용되는 문자는 모두 ASCII 문자임**
- **▪ 프로그램은 입력을 무한히 기다리며, 사용자 입력이 "PA1EXIT"일 때 종료됨**

Printing output

- ▪ 입력된 검색어가 파일 내에 여러 번 등장하는 경우
  - 1, 3: 같은 줄에 여러 번 등장하더라도 모든 위치를 출력해야 함
  - 2, 4: 여러 번 등장하더라도 줄 번호는 한 번만 출력해야 함
  - 각 위치 또는 줄 번호 뒤에는 공백 한 칸을 추가하고, 검색이 끝나면 줄바꿈(개행) 해야 함
    - » 예:
      $"he is“

      955:33 2315:42 5885:22↵
  - 마지막 항목 뒤에도 공백을 추가하고 줄바꿈하는 것이 원칙
  - 출력 결과는 순서대로 정렬되어야 함
    - » 예: 1:0 1:2 4:8 6:13 9:8처럼 정렬된 순서로 출력되어야 하며,
      9:8 1:2 4:8 1:0 6:13처럼 뒤섞인 순서는 허용되지 않음
- 모든 입력 질의에 대해 출력 결과가 있어야 함
  - 검색 결과가 없더라도 출력 라인은 반드시 존재해야 하며, 결과가 없을 경우 빈 출력을 해야 함
  - 예: "1:0\n\n1:0\n" 과 같은 형식에서, 두 번째 줄은 결과가 없음을 의미하는 빈 출력 줄임
  - 즉, stdout의 n번째 줄은 n번째 질의에 대한 결과를 나타내야 함

Notes on the input text file

- **입력 텍스트 파일의 파일명은 명령줄 인자(argv[1])로 전달됨**
- **텍스트 파일 내의 모든 문자는 ASCII 문자로 구성됨**
- **파일의 전체 길이는 시스템 메모리보다 훨씬 클 수 있음**
  - 즉, 파일 전체를 한 번에 메모리로 불러오는 것은 불가능함
- **단, 한 줄의 길이 또는 한 줄 내 단어의 길이는 시스템 메모리보다 작음 → 한 줄씩 또는 한 단어 단위로 처리하는 것은 가능함**

Scoring Policy

- 총점은 100점 만점이며, 위에서 설명한 형식을 반드시 준수해야 함 형식을 지키지 않을 경우, 점수가 부여되지 않음
- 마감 기한 이후 제출 시 하루당 10점씩 감점되며, 최대 4일까지만 감점 적용됨
- 점수는 테스트 케이스 통과 여부에 따라 부여됨
- 여러 개의 입력 파일을 사용해 채점을 진행하며, 예시로 제공된 입력 파일도 포함됨
- 일부 입력 파일은 10GB를 초과할 수 있음
- 테스트 케이스는 약 10초 이내에 실행 완료되어야 하며, 5분 이상 소요될 경우 해당 케이스는 0점 처리됨
- 채점 서버의 메모리는 1GB로 가정됨

Restrictions

- **제한 사항**
  - 프로젝트는 반드시 Linux 환경에서 컴파일 및 실행되어야 함
  - 지정된 헤더 파일 외의 헤더를 사용할 경우 **0점 처리**될 수 있음
    - » 직접 구현한 헤더 파일이 최소 1개 이상 포함되어야 함 (필수)
    - » **`unistd.h, fcntl.h, stdlib.h, sys/types.h, sys/stat.h, errno.h, stdint.h`**
  - 본 과제는 개인 과제 입니다.
    - » 서로 아이디어를 논의하는 것은 가능하지만, 소스 코드는 반드시 본인이 직접 작성해야 하는 것을 원칙으로 합니다.
  - Makefile은 반드시 **pa1** 이름의 실행 파일을 생성해야 함
    - » **그렇지 않을 경우, 0점 처리될 수 있음**

Submission

- 프로젝트 디렉토리 구조는 아래와 같아야 함

```
pa1 (directory)
- *.c (C code files)
- *.h (Header files)
- Makefile
```

- pa1 디렉토리를 압축하여 제출해야 하며, 압축 파일 이름은 다음 형식을 따라야 함. [학번]_pa1.tar.zst
- 완성된 압축 파일은 마감 기한 전에 iCampus에 제출해야 함
- 마감일: 2026년 4월 5일 (일) 23:59

Example

```
$ ./pa1 500-Days-of-Summer_s.txt
500 days
6 6419
he is
50 1039 1822 1955 2256 2315 3494 3503 4353 4360 4445 4831 5101 5885 6325
"he is"
1955:33 2315:42 5885:22
he*is
2315 4445 5101
she he
1370 1512 1513 3423 3473 3478 3550 4255 4510 4515 5413 5672 6154 6188
loved
106:30 1122:9 1150:24 3961:25 4739:17 5921:28

…
```

Example

```
$ ./pa1 500-Days-of-Summer_s.txt > result.out
he
hey
she
tom
summer is
_summer is"
landscape
together
we*her
no much
immediately
no*much
quarterback
we
PA1EXIT

$ diff -bsq result.out answer.out
Files result.out and answer.out are identical
$ diff result.out answer.out
$
```